

Weather and Daily Step Count Analysis

Final Report

DSA 210 - Introduction to Data Science

Alperen Guldur

GitHub Repository: <https://github.com/alperenguldur/-DSA210-DataScienceProject>

Project Summary

In this project, I analyzed whether daily weather conditions are related to my daily walking activity. The step data comes from Apple Health and the weather data comes from Open-Meteo. I cleaned both datasets, merged them by date, explored the data with graphs, tested simple hypotheses, and then applied regression models to predict daily step count.

The main result is simple: in this dataset, weather variables alone do not explain my steps very well. Rainy days and non-rainy days were not statistically different, and temperature and precipitation had weak correlations with steps. The machine learning results also show that more personal variables, such as class days, exams, sleep, and daily routine, would probably be needed for a better model.

Project Requirements Covered

Requirement	Where it is covered
GitHub repository	The project repository link is included in this report.
Project proposal idea	The research question, motivation, datasets, and plan are explained.
Data collection, EDA, and hypothesis tests	The report includes data cleaning, visualizations, correlations, and statistical tests.
Machine learning methods	Linear Regression, Ridge Regression, Decision Tree Regression, KNN Regression, and a baseline model are included.
Final report and code	This PDF summarizes the project and the full combined code is included in the appendix.

1. Introduction

My research question is: Is there a meaningful relationship between daily weather conditions and daily step count?

The target variable is daily steps. The main explanatory variables are average temperature, precipitation, weather code, rainy day indicator, weekday, weekend indicator, and month.

2. Data Collection and Cleaning

The Apple Health file had 104 rows and the weather file had 102 rows. After cleaning and merging by date, the final dataset had 102 daily observations from 2026-01-01 to 2026-04-12.

Dataset	Size	Notes
Apple Health steps	104 rows	Daily total step count
Open-Meteo weather	102 rows	Daily temperature and precipitation
Merged dataset	102 rows	Final dataset used for analysis

I converted the date columns to datetime format, converted numerical columns, removed invalid rows, and created new variables such as rainy, weekday, is_weekend, day_of_week, and month. The final modeling variables had no missing values.

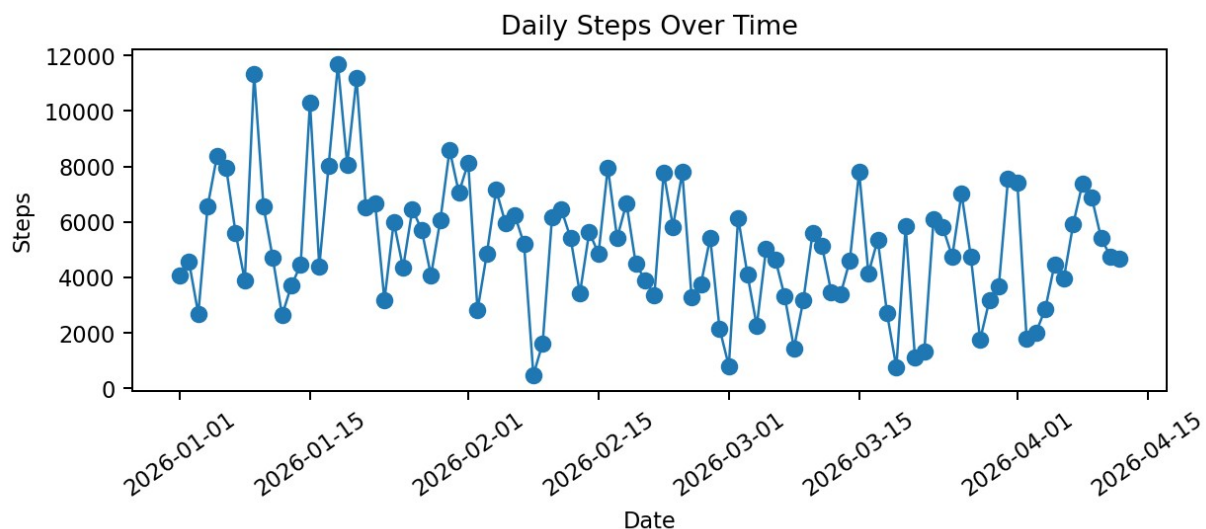
3. Descriptive Statistics

Variable	Mean	Std. Dev.	Minimum	Maximum
Steps	5,115.93	2,289.21	478	11,680
Temperature (C)	8.87	3.24	0.40	15.43
Precipitation (mm)	0.97	1.70	0.00	9.10

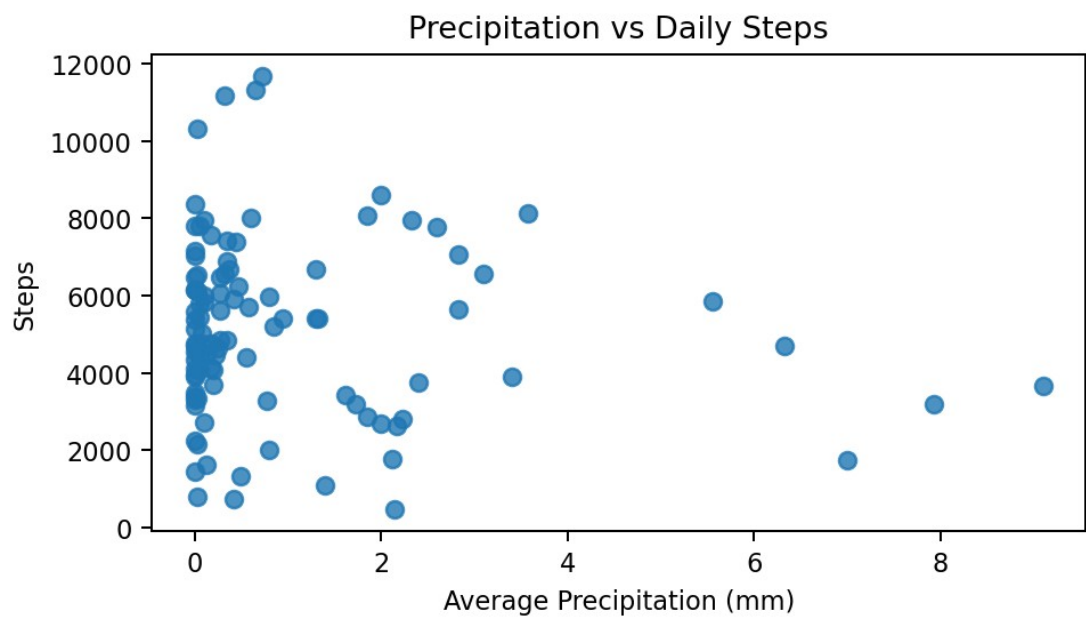
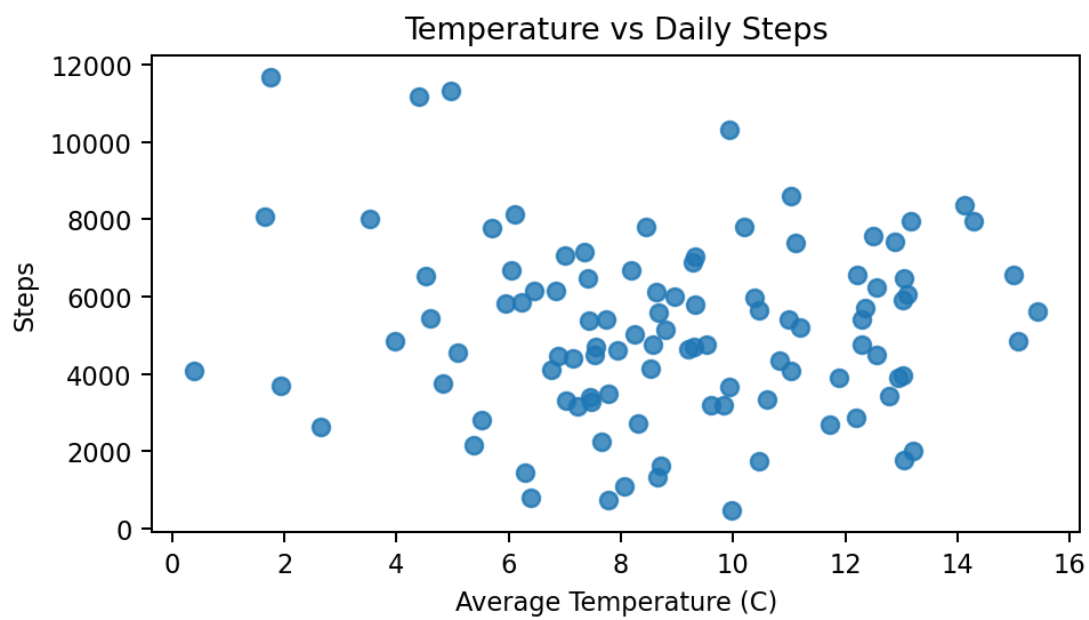
There were 79 rainy days and 23 non-rainy days. The average step count was about 5,116 steps per day.

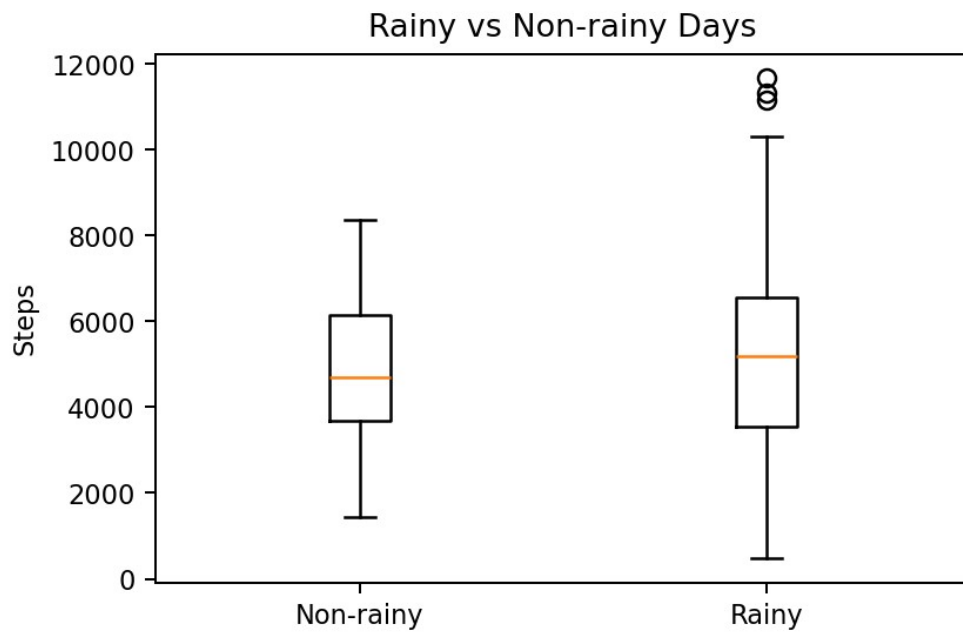
4. Exploratory Data Analysis

The graph below shows daily steps over time. The values change a lot from day to day, so step count seems to depend on more than only weather.

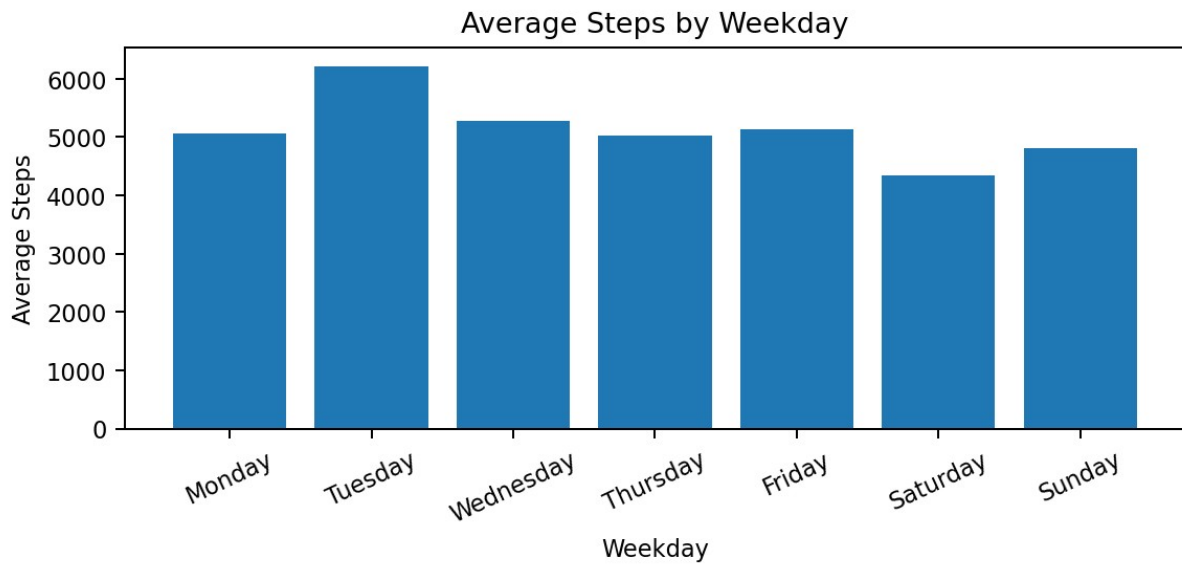


The scatter plots show weak relationships between weather variables and steps.



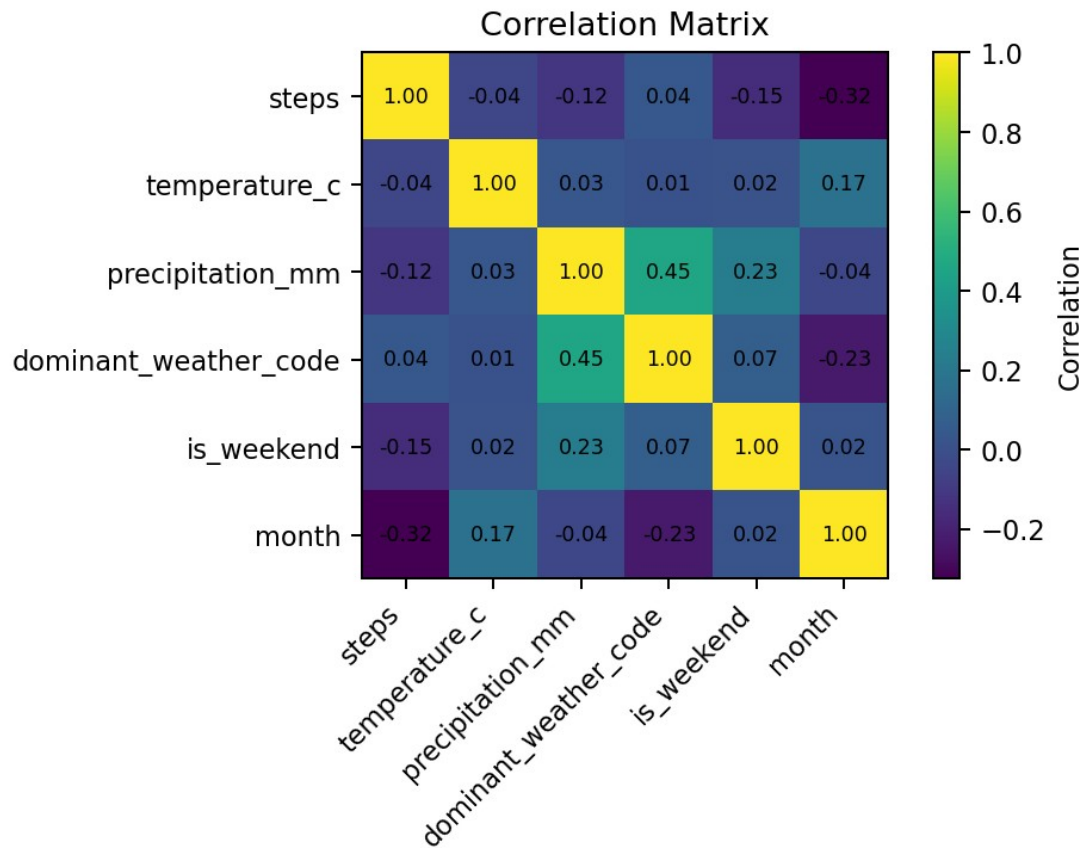


Average steps on non-rainy days were 4,893.78. Average steps on rainy days were 5,180.61. Rainy days were slightly higher by 286.82 steps, but this difference is small.



Weekday	Average steps
Monday	5,066
Tuesday	6,221
Wednesday	5,278
Thursday	5,035
Friday	5,140
Saturday	4,336
Sunday	4,816

5. Correlation Analysis



Variable	Correlation with steps
month	-0.3248
is_weekend	-0.1529
precipitation_mm	-0.1166
temperature_c	-0.0424
dominant_weather_code	0.0423

The weather correlations are weak. Precipitation has a small positive correlation with steps and temperature is almost zero. This means weather alone does not look like a strong explanation for daily steps.

6. Hypothesis Testing

Hypothesis 1: Rainy vs. Non-rainy Days

H0: Average steps are the same on rainy and non-rainy days. H1: Average steps are different on rainy and non-rainy days.

Test	Statistic	P-value	Result
Welch t-test	0.6272	0.5334	Not significant
Mann-Whitney U test	972.0000	0.6139	Not significant

Both p-values are higher than 0.05, so I fail to reject H0. There is not enough evidence that rainy and non-rainy days have different step counts.

Hypothesis 2: Weather Variables and Steps

Feature	Pearson r	Pearson p-value	Spearman rho	Spearman p-value
Temperature	-0.0424	0.6725	0.0234	0.8154
Precipitation	-0.1166	0.2434	-0.0042	0.9664

All p-values are above 0.05, so the correlations are not statistically significant.

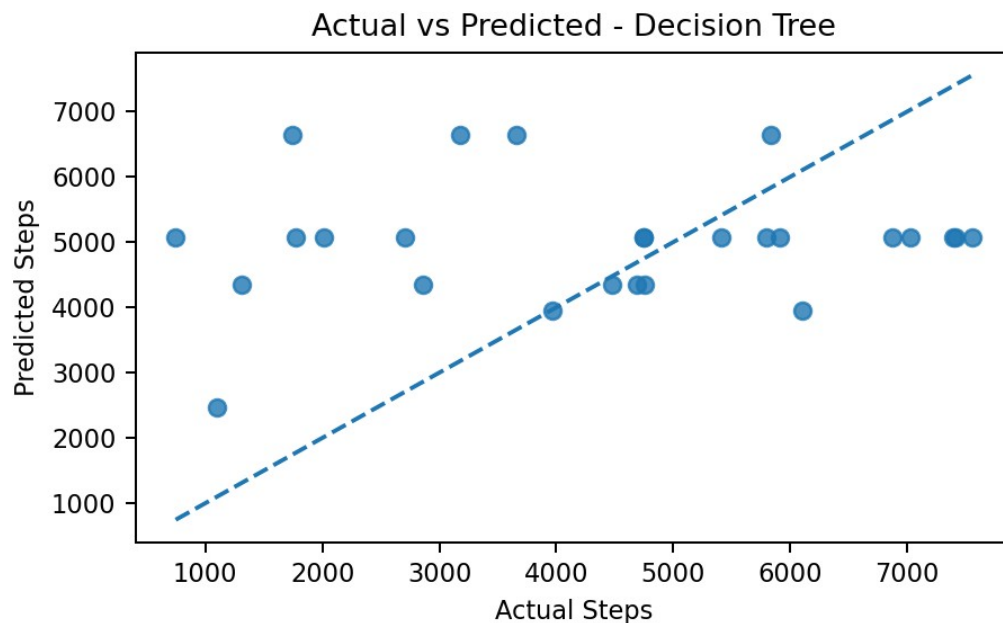
7. Machine Learning

The prediction task is regression because the target variable is daily step count. I used a baseline mean model, Linear Regression, Ridge Regression, Decision Tree Regression, and KNN Regression. These models are suitable for this type of introductory data science project.

I used a chronological split: the first 75% of the data for training and the last 25% for testing. The test period is from 2026-03-18 to 2026-04-12.

Model	MAE	RMSE	R2
Decision Tree	1,833.29	2,271.58	-0.191
Baseline Mean	1,887.56	2,305.66	-0.227
KNN Regression	2,023.72	2,354.07	-0.279
Ridge Regression	2,125.81	2,446.70	-0.381
Linear Regression	2,147.39	2,469.25	-0.407

The best test MAE was achieved by Decision Tree. Still, the R2 values are weak, so the models are not very reliable for predicting daily steps from only weather and calendar variables.

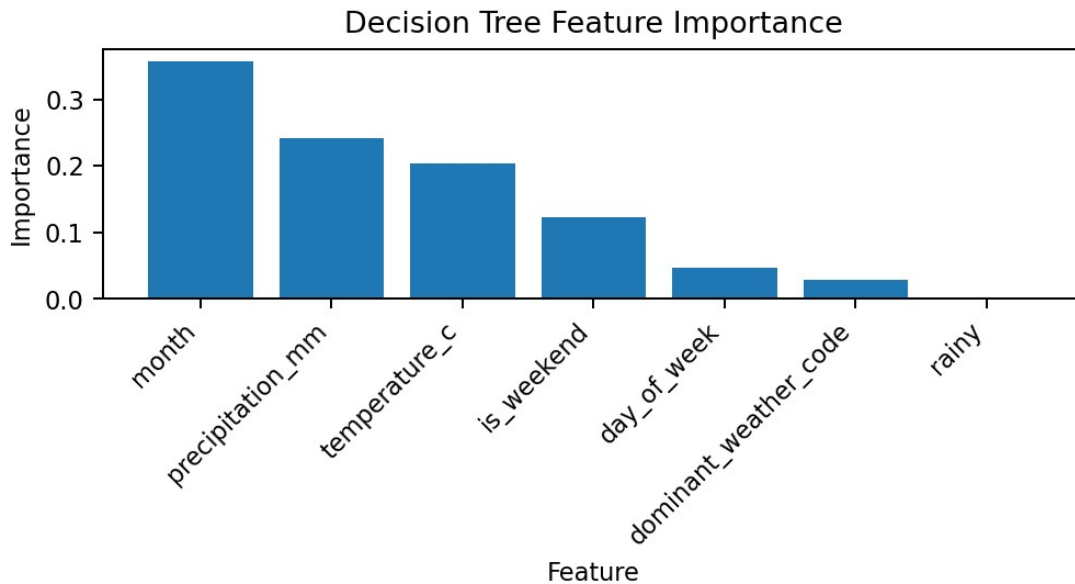


Cross-validation

Model	CV MAE Mean	CV RMSE Mean	CV R2 Mean
Baseline Mean	1,798.10	2,232.22	-0.006
Ridge Regression	1,824.99	2,267.89	-0.082
Linear Regression	1,828.43	2,271.51	-0.086
KNN Regression	1,895.92	2,341.79	-0.129
Decision Tree	2,081.50	2,636.00	-0.466

Cross-validation also shows that the prediction performance is limited. This supports the idea that more personal variables are needed.

8. Model Interpretation



Feature	Decision Tree importance
month	0.358
precipitation_mm	0.241
temperature_c	0.204
is_weekend	0.122
day_of_week	0.047
dominant_weather_code	0.028
rainy	0.000

The feature importance values show how the Decision Tree used the variables. This is not a causal result. It only helps interpret the model.

9. Conclusion

Overall, I did not find strong evidence that weather explains my daily step count. Rainy days had slightly higher average steps, but this was not statistically significant. Temperature and precipitation correlations were weak and insignificant. The machine learning models also had weak predictive performance.

My conclusion is that daily routine probably matters more than daily weather averages. In future work, I would add more personal variables such as exam days, class schedule, sleep, transport mode, and location.

10. Limitations and Future Work

- The dataset is small with 102 daily observations.
- Daily averages may hide hourly weather effects.
- Personal schedule, exams, sleep, and location are not included.
- Correlation does not prove causation.
- More months of data would make the analysis stronger.

11. Repository Files

GitHub Repository URL: <https://github.com/alperenguldur/-DSA210-DataScienceProject>

File	Purpose
Final report PDF	Main report with results and code appendix
Notebook / Python code	Runs the full analysis
apple-steps-since-2026-01-01.csv	Step-count data
open-meteo-daily-averages.csv	Weather data
cleaned_weather_steps_merged.csv	Cleaned merged dataset
README.md	Repository explanation
requirements.txt	Needed Python packages

Appendix A - Complete Combined Code

Below is the combined code used for the project. It includes loading the files, cleaning, EDA, hypothesis testing, machine learning, cross-validation, and saving the cleaned dataset.

```
# Weather and Daily Step Count Analysis - Combined Code
```

```
from pathlib import Path
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import ttest_ind, mannwhitneyu, pearsonr, spearmanr

from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import KFold, cross_validate
from sklearn.neighbors import KNeighborsRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeRegressor

warnings.filterwarnings("ignore")

# 1. Load data
STEP_FILE = Path("apple-steps-since-2026-01-01.csv")
WEATHER_FILE = Path("open-meteo-daily-averages.csv")

def read_csv_safely(path: Path) -> pd.DataFrame:
    if not path.exists():
        raise FileNotFoundError(f"Could not find {path}. Make sure it is in the same folder as the notebook.")
    return pd.read_csv(path, sep=None, engine="python")

steps_raw = read_csv_safely(STEP_FILE)
weather_raw = read_csv_safely(WEATHER_FILE)

print("Steps data shape:", steps_raw.shape)
print("Weather data shape:", weather_raw.shape)
print("Steps columns:", list(steps_raw.columns))
print("Weather columns:", list(weather_raw.columns))

# 2. Clean and merge data
steps_df = steps_raw.copy()
weather_df = weather_raw.copy()

steps_df = steps_df.rename(columns={"total_steps": "steps"})
weather_df = weather_df.rename(columns={"avg_temperature_2m_c": "temperature_c",
                                       "avg_precipitation_mm": "precipitation_mm"})
})

required_steps_cols = {"date", "steps"}
required_weather_cols = {"date", "temperature_c", "precipitation_mm", "dominant_weather_code"}
```



```

missing_steps = required_steps_cols - set(steps_df.columns)
missing_weather = required_weather_cols - set(weather_df.columns)

if missing_steps:
    raise ValueError(f"Missing required step columns: {missing_steps}")
if missing_weather:
    raise ValueError(f"Missing required weather columns: {missing_weather}")

steps_df["date"] = pd.to_datetime(steps_df["date"], errors="coerce")
weather_df["date"] = pd.to_datetime(weather_df["date"], errors="coerce")

steps_df["steps"] = pd.to_numeric(steps_df["steps"], errors="coerce")
weather_df["temperature_c"] = pd.to_numeric(weather_df["temperature_c"], errors="coerce")
weather_df["precipitation_mm"] = pd.to_numeric(weather_df["precipitation_mm"], errors="coerce")
weather_df["dominant_weather_code"] = pd.to_numeric(weather_df["dominant_weather_code"], errors="coerce")

steps_df = steps_df.dropna(subset=["date", "steps"])
weather_df = weather_df.dropna(subset=["date", "temperature_c", "precipitation_mm", "dominant_weather_code"])

df = pd.merge(steps_df, weather_df, on="date", how="inner")
df = df.sort_values("date").reset_index(drop=True)

df["rainy"] = df["precipitation_mm"] > 0
df["weekday"] = df["date"].dt.day_name()
df["day_of_week"] = df["date"].dt.dayofweek
df["is_weekend"] = df["day_of_week"].isin([5, 6]).astype(int)
df["month"] = df["date"].dt.month

print("Merged data shape:", df.shape)
print("Date range:", df["date"].min().date(), "to", df["date"].max().date())
print(df.isna().sum())

# 3. Data overview
summary_cols = ["steps", "temperature_c", "precipitation_mm", "dominant_weather_code", "is_weekend", "month"]
print(df[summary_cols].describe().T)

rainy_steps = df.loc[df["rainy"], "steps"]
non_rainy_steps = df.loc[~df["rainy"], "steps"]

print("Number of rainy days:", int(df["rainy"].sum()))
print("Number of non-rainy days:", int(~df["rainy"].sum()))
print("Mean steps on non-rainy days:", round(non_rainy_steps.mean(), 2))
print("Mean steps on rainy days:", round(rainy_steps.mean(), 2))

# 4. EDA graphs
plt.figure(figsize=(12, 5))
plt.plot(df["date"], df["steps"], marker="o")
plt.title("Daily Steps Over Time")
plt.xlabel("Date")
plt.ylabel("Steps")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.scatter(df["temperature_c"], df["steps"], alpha=0.8)
plt.title("Temperature vs Daily Steps")
plt.xlabel("Average Temperature (C)")
plt.ylabel("Steps")
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.scatter(df["precipitation_mm"], df["steps"], alpha=0.8)
plt.title("Precipitation vs Daily Steps")
plt.xlabel("Average Precipitation (mm)")
plt.ylabel("Steps")
plt.tight_layout()
plt.show()

plt.figure(figsize=(7, 5))
plt.boxplot([non_rainy_steps, rainy_steps], labels=["Non-rainy", "Rainy"])
plt.title("Step Counts on Rainy vs Non-rainy Days")

```

```

plt.ylabel("Steps")
plt.tight_layout()
plt.show()

weekday_order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
weekday_steps = df.groupby("weekday")["steps"].mean().reindex(weekday_order)

plt.figure(figsize=(9, 5))
plt.bar(weekday_steps.index, weekday_steps.values)
plt.title("Average Steps by Weekday")
plt.xlabel("Weekday")
plt.ylabel("Average Steps")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

corr_cols = ["steps", "temperature_c", "precipitation_mm", "dominant_weather_code", "is_weekend", "month"]
corr = df[corr_cols].corr(numeric_only=True)
print(corr)

plt.figure(figsize=(8, 6))
plt.imshow(corr)
plt.colorbar(label="Correlation")
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45, ha="right")
plt.yticks(range(len(corr.columns)), corr.columns)
for i in range(len(corr.columns)):
    for j in range(len(corr.columns)):
        plt.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center")
plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()

# 5. Hypothesis tests
# Hypothesis 1: Rainy and non-rainy days have different average steps.
t_stat, p_value_t = ttest_ind(rainy_steps, non_rainy_steps, equal_var=False)
u_stat, p_value_u = mannwhitneyu(rainy_steps, non_rainy_steps, alternative="two-sided")

print("Welch t-test")
print("T-statistic:", round(t_stat, 4))
print("P-value:", round(p_value_t, 4))
print("Mann-Whitney U test")
print("U-statistic:", round(u_stat, 4))
print("P-value:", round(p_value_u, 4))

# Hypothesis 2: Weather variables are correlated with daily steps.
for feature in ["temperature_c", "precipitation_mm"]:
    pearson_corr, pearson_p = pearsonr(df[feature], df["steps"])
    spearman_corr, spearman_p = spearmanr(df[feature], df["steps"])
    print(f"Feature: {feature}")
    print(f" Pearson r = {pearson_corr:.4f}, p-value = {pearson_p:.4f}")
    print(f" Spearman rho = {spearman_corr:.4f}, p-value = {spearman_p:.4f}")

# 6. Machine learning
feature_cols = [
    "temperature_c",
    "precipitation_mm",
    "dominant_weather_code",
    "rainy",
    "is_weekend",
    "day_of_week",
    "month",
]
target_col = "steps"

model_df = df.dropna(subset=feature_cols + [target_col]).copy()
model_df["rainy"] = model_df["rainy"].astype(int)

X = model_df[feature_cols]
y = model_df[target_col]

split_index = int(len(model_df) * 0.75)
X_train, X_test = X.iloc[:split_index], X.iloc[split_index:]
y_train, y_test = y.iloc[:split_index], y.iloc[split_index:]

linear_preprocessor = ColumnTransformer(
    transformers=[

```

```

        ("num", Pipeline(steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler())
        ]), feature_cols)
    ]
)

tree_preprocessor = ColumnTransformer(
    transformers=[("num", SimpleImputer(strategy="median"), feature_cols)]
)

models = {
    "Baseline Mean": Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("model", DummyRegressor(strategy="mean"))
    ]),
    "Linear Regression": Pipeline(steps=[
        ("preprocess", linear_preprocessor),
        ("model", LinearRegression())
    ]),
    "Ridge Regression": Pipeline(steps=[
        ("preprocess", linear_preprocessor),
        ("model", Ridge(alpha=1.0))
    ]),
    "Decision Tree": Pipeline(steps=[
        ("preprocess", tree_preprocessor),
        ("model", DecisionTreeRegressor(max_depth=4, min_samples_leaf=5, random_state=42))
    ]),
    "KNN Regression": Pipeline(steps=[
        ("preprocess", linear_preprocessor),
        ("model", KNeighborsRegressor(n_neighbors=5))
    ]),
}

def evaluate_model(name, model, X_train, X_test, y_train, y_test):
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, preds))
    return {
        "model": name,
        "MAE": mean_absolute_error(y_test, preds),
        "RMSE": rmse,
        "R2": r2_score(y_test, preds),
    }, preds

results = []
predictions = {}
for name, model in models.items():
    result, preds = evaluate_model(name, model, X_train, X_test, y_train, y_test)
    results.append(result)
    predictions[name] = preds

results_df = pd.DataFrame(results).sort_values("MAE")
print(results_df)

best_model = results_df.iloc[0]["model"]
plt.figure(figsize=(8, 5))
plt.scatter(y_test, predictions[best_model], alpha=0.8)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], linestyle="--")
plt.title(f"Actual vs Predicted Steps - {best_model}")
plt.xlabel("Actual Steps")
plt.ylabel("Predicted Steps")
plt.tight_layout()
plt.show()

comparison_df = pd.DataFrame({
    "date": model_df["date"].iloc[split_index:].values,
    "actual_steps": y_test.values,
})
for name, preds in predictions.items():
    comparison_df[f"pred_{name}"] = np.round(preds, 0).astype(int)
print(comparison_df.head(10))

# 7. Cross-validation
cv = KFold(n_splits=5, shuffle=True, random_state=42)
cv_results = []
for name, model in models.items():
    scores = cross_validate(

```

```

        model,
        X,
        y,
        cv=cv,
        scoring={
            "MAE": "neg_mean_absolute_error",
            "RMSE": "neg_root_mean_squared_error",
            "R2": "r2",
        },
        return_train_score=False,
    )
    cv_results.append({
        "model": name,
        "CV_MAE_mean": -scores["test_MAE"].mean(),
        "CV_MAE_std": scores["test_MAE"].std(),
        "CV_RMSE_mean": -scores["test_RMSE"].mean(),
        "CV_R2_mean": scores["test_R2"].mean(),
    })

cv_results_df = pd.DataFrame(cv_results).sort_values("CV_MAE_mean")
print(cv_results_df)

# 8. Decision Tree feature importance
tree_model = models["Decision Tree"]
tree_model.fit(X_train, y_train)

importances = tree_model.named_steps["model"].feature_importances_
importance_df = pd.DataFrame({
    "feature": feature_cols,
    "importance": importances,
}).sort_values("importance", ascending=False)
print(importance_df)

plt.figure(figsize=(8, 5))
plt.bar(importance_df["feature"], importance_df["importance"])
plt.title("Decision Tree Feature Importance")
plt.xlabel("Feature")
plt.ylabel("Importance")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

# 9. Save cleaned dataset
df.to_csv("cleaned_weather_steps_merged.csv", index=False)
print("Saved cleaned merged dataset to cleaned_weather_steps_merged.csv")

```